

**ТОП 30
ВОПРОСОВ
НА DATA
ENGINEER
ЭТОЙ
ОСЕНЬЮ**

ОБО МНЕ

Я – Алексей Богатырев. В IT 5 лет. Работал в топ банках по типу Райффайзенбанк и Альфа банк.

Сейчас работаю техлидом в Денум. Занимаюсь менторством и вкатываю людей в data engineering.

Походил по собесам и вот топ 30 вопросов и ответов что спрашивают на собеседах этой осенью:

SQL

1. ПОРЯДОК ВЫПОЛНЕНИЯ КОМАНД

FROM источники данных, JOIN'ы

WHERE фильтрация строк

GROUP BY группировка

HAVING фильтрация групп

SELECT выборка и вычисления

DISTINCT удаление дубликатов

ORDER BY сортировка

LIMIT / **OFFSET** ограничение вывода

2. ФИЗИЧЕСКИЕ JOIN

Nested Loop Join Перебирает строки внешней таблицы, для каждой ищет в другой.

Hash Join Строит хеш-таблицу по одной таблице, пробегаает вторую и ищет по хешу.

Merge Join (Sort-Merge Join) Обе таблицы должны быть отсортированы по ключу.

3. КАК ОПТИМИЗИРОВАТЬ ЗАПРОС

Если запрос медленный:

1. Добавь индексы по полям фильтрации, джойна и сортировки. Всегда выбирай только нужные поля без SELECT WHERE до JOIN, LIMIT до ORDER BY, по возможности.
2. Проверь, можно ли агрегацию сделать на меньшем объёме.
3. Используй оконные функции, если не нужна группировка.
4. Всегда смотри план запроса. Найдёшь слабые места: full scan, nested loop, лишние сортировки.

4. АГРЕГАЦИИ VS ГРУППИРОВКА

Агрегатные функции Преобразуют много строк в одно значение: Работают сами по себе или с GROUP BY.

Группировка (GROUP BY) Делит строки на группы, а агрегатные функции применяются к каждой группе.

5. КАКИЕ ЗНАЕШЬ ОКОННЫЕ ФУНКЦИИ

Оконная функция это функция, которая вычисляется по набору строк ("окну"), но не агрегирует их в одну строку, как GROUP BY, а оставляет каждую строку.

Примеры использования:

ROW_NUMBER() для нумерации внутри группы, например, выделить первую запись по времени.

LAG(), **LEAD()** посмотреть предыдущую/следующую строку (например, для расчёта разницы во времени).

SUM() OVER(PARTITION BY...) посчитать накопительный итог по пользователю или категории

PYTHON

1. ОСНОВНЫЕ КОЛЛЕКЦИИ, ИЗМЕНЯЕМЫЕ И НЕИЗМЕНЯЕМЫЕ ТИПЫ

Dict – изменяемый, ключ-значение. ключ уникален

Set – изменяемый, без порядка, только уникальные элементы

List – изменяемый, упорядоченный

Tuple – неизменяемый, упорядоченный

Нельзя изменить после создания:

int – целые числа

float – числа с плавающей точкой

bool – логические значения

str – строки

tuple – кортежи

Можно изменять после создания:

list – списки

dict – словари

set – множества

2. ДЕКОРАТОР

Декоратор это функция, которая принимает другую функцию и расширяет её функционал, не изменяя исходный код.

3. КОНТЕКСТНЫЕ МЕНЕДЖЕРЫ

Специальный объект для управления ресурсами, который потом с гарантией освобождает эти ресурсы

Как работает:

Объект должен реализовывать методы:

__enter__() – вход в контекст, возвращает ресурс

__exit__() – выход из контекста, освобождение ресурса

4. ГЕНЕРАТОР И ИТЕРАТОР

Итератор (Iterator)

Объект, который реализует:

`__iter__()` – возвращает сам объект

`__next__()` – возвращает следующий элемент

Все значения хранятся в памяти. может не хватить памяти и приложение упадет

Генератор (Generator)

Специальный тип итератора, создаваемый с помощью:

Функции с `yield`

Генераторных выражений

Экономит память. в памяти хранит только текущее значение

5. КАК УСТРОЕН СЛОВАРЬ И КАК ОН РЕШАЕТ КОЛЛИЗИИ

Как устроен:

1. **Хеш-функция**: Для ключа вычисляется хеш (через `hash()`).

2. **Индекс**: На основе хеша вычисляется индекс ячейки в массиве (с помощью маски).

3. **Ячейка (bucket)**: Каждая ячейка хранит ссылку на запись:
[hash, key, value].

Решение коллизий (Collision Resolution)

Метод открытой адресации (Open Addressing):

Если ячейка занята ищется следующая свободная (линейное пробирование).

В современных версиях Python используется более сложная версия (например, псевдослучайное пробирование).

Что происходит при добавлении:

1. Вычисляется хеш ключа.
2. Находится ячейка по индексу.
3. Если ячейка занята ищется следующая свободная.
4. Данные сохраняются в первую свободную ячейку.

При поиске:

1. Вычисляется хеш ключа.
2. Проверяется ячейка по индексу.
3. Если ключ не совпадает проверяются следующие ячейки, пока не найдётся совпадение или пустая ячейка.

ТЕОРИЯ DWH

1. SCD

SCD (Slowly Changing Dimension) это способ хранения изменений в размерностях со временем.

SCD0 – Значения фиксируются один раз и не меняются

SCD1 – Новое значение перезаписывает старое

SCD2 – Добавляется новая строка при каждом изменении (храним историчность)

SCD3 – Храним только 1 предыдущее значение в той же строке

Примечание: Есть еще SCD4–SCD6 но почти не используется. На собеседах не спрашивают.

2. OLTP VS OLAP

OLTP (Online Transaction Processing):

Частые короткие транзакции (INSERT, UPDATE, SELECT)

Чаще нормализованная схема (3NF)

Чаще всего источник данных

OLAP (Online Analytical Processing):

Сложные аналитические запросы (JOIN, GROUP BY, агрегации)

Денормализованная схема (звезда/снежинка)

Высокая пропускная способность, оптимизация под чтение

Чаще всего приемник данных (DWH)

3. DATA VAULT VS КИМБАЛЛ

Метод Кимбалла – только денормализация. Используется звезда. Делаем витрину только для одного домена. Не для всей компании. Сначала прям классно, но чем больше доменов бизнеса нужно добавлять тем затратнее

Data Vault – это подход к построению DWH, ориентированный на историчность, масштабируемость и интеграцию данных из разных источников.

Компоненты:

Hub – таблица с бизнес-ключами, значения уникальные и неизменяемые и нет бизнес-атрибутов. Хранит бизнес-ключ, дату загрузки, источник данных

Link – связь между Hub. Хранит многие-ко-многим, версии и источники.

Satellite таблица с атрибутами и историей. **Привязан к Hub или Link.**

Только здесь хранится историчность (SCD2).

4. ИНДЕКСЫ В БД

Индекс – структура данных, которая ускоряет поиск, фильтрацию и сортировку.

Применяется когда:

На столбцах которые часто фильтруют данные

Столбцы в WHERE условиях – для ускорения соединений таблиц

Столбцы в JOIN условиях – для ускорения соединений таблиц

Столбцы в ORDER BY – особенно если часто требуется сортировка

Столбцы в GROUP BY – для ускорения агрегации

Когда таблица большая

Виды индексов:

Кластерный индекс

Физически сортирует строки таблицы по значению индекса.

В таблице можно быть только один кластерный индекс.

Хранится в одном файле с данными.

Некластерный индекс

Хранит ссылки на физическое расположение строк и ключевые значения в таблице.

В таблице может быть много некластерных индексов.

Хранится отдельно от данных (в другом файле).

5. ACID

Atomicity (атомарность) транзакция выполняется целиком или откатывается.

Consistency (согласованность) данные до и после выполнения транзакции должны быть согласованы (например, для каждого внешнего ключа должен быть первичный ключ).

Isolation (изолированность) все транзакции запускаются в отдельно и не влияют друг на друга.

Durability (долговечность) Если транзакция зафиксирована через COMMIT то ее результат не потеряется даже при отключении питания.

AIRFLOW

1. ОСНОВНЫЕ КОМПОНЕНТЫ

1. Web Server

UI для мониторинга и управления

Показывает DAGs, логи, статусы задач

2. Scheduler

Мозг системы – планирует и запускает задачи
Определяет когда выполнять DAGs и таски
Работает с метаданными в БД

3. Executor

Исполнитель – как запускать задачи:
SequentialExecutor – последовательно (dev)
LocalExecutor – мультипроцессорный
CeleryExecutor – распределенный
KubernetesExecutor – в K8s pods

4. Workers

Воркеры – выполняют задачи (для Celery/K8s)
Масштабируются горизонтально

5. Metadata Database

БД (PostgreSQL/MySQL) – хранит:
DAGs метаданные
Статусы задач
XCom, переменные, connections

2. ВИДЫ EXECUTORS

1. SequentialExecutor

Для разработки и тестирования
Запускает задачи последовательно
Не требует БД кроме SQLite

2. LocalExecutor

Мультипроцессорный на одной машине
Подходит для небольших продакшенов
Использует БД (PostgreSQL/MySQL)

3. CeleryExecutor

Распределенный через Celery
Масштабируемый – multiple workers
Требует брокера сообщений (Redis/RabbitMQ)

4. KubernetesExecutor

Запускает задачи в отдельных Kubernetes Pods
Автоскейлинг – ресурсы по требованию
Для больших кластеров

5. DaskExecutor

Распределенные вычисления через Dask
Для вычислительно сложных задач

3. XCOM

Cross-communication – механизм обмена данными между задачами.

Для чего: Передача маленьких данных между задачами

Ключи, флаги, статусы, небольшие объекты

Не для больших файлов/данных

Как работает:

Данные хранятся в Metadata Database

Сериализация через Base64

Ограничение размера (~64KB в зависимости от БД)

4. СЕНСОРЫ. КАКИЕ ИСПОЛЬЗОВАЛ?

Сенсор (Sensor)

Проверяет условие (файл появился, данные готовы)

Наследуется от BaseSensorOperator

Работает в режиме poke или reschedule

5. ЧТО ТАКОЕ ХУК? ЧЕМ ХУК ОТЛИЧАЕТСЯ ОТ ОПЕРАТОРА?

Хук – Соединение с внешней системой

Пример: PostgresHook, S3Hook

оператор

Конкретное действие или задача

Пример: PythonOperator, BashOperator

Выполняет работу, использует хуки внутри

Отличие:

Хук = подключение, Оператор = действие

Операторы используют хуки для доступа к системам и выполнения работы.

CLICKHOUSE

1. СЕМЕЙСТВО MERGETREE

MergeTree базовый и самый важный. Поддерживает партиционирование, сортировку, индекс

ReplacingMergeTree удаляет дубликаты. Можно указать version для выбора последней строки

AggregatingMergeTree Хранит агрегированные состояния (AggregateFunction(...))

SummingMergeTree Автоматически агрегирует строки с одинаковыми ключами

CollapsingMergeTree Удаляет пары строк с противоположными знаками sign (+1 / -1)

VersionedCollapsingMergeTree Как Collapsing, но учитывает version оставляет последнюю актуальную строку

2. ИНДЕКСЫ

Отдельно задаются для ускорения фильтрации по неключевым колонкам. Помогают пропустить блоки, в которых точно нет нужных данных.

Типы:

minmax – Сохраняет min/max значений на блок

set – Сохраняет set уникальных значений

bloom_filter – Быстрый поиск по exact match

tokenbf_v1 – Полнотекстовый поиск по словам

ngrambf_v1 – Полнотекст по подстрокам (n-граммам)

3. СТРОКОВАЯ VS СТОЛБЦОВАЯ БД

Строковая БД – данные на диске хранятся блоками. Даже если тебе нужна только одна колонка, ты не сможешь вытащить только её.

В широких таблицах (много столбцов) будет очень долго.

Столбцовая БД – данные хранятся по колонкам, а не по строкам. Каждая колонка хранится отдельно на диске. А значения в колонке хранятся последовательно.

Из-за того что база загружает только нужные данные – она идеально подходит для аналитических запросов.

4. JOIN

JOIN в ClickHouse не по строчке, а блочно

Всё работает в потоках и блоками, для максимальной скорости.

Нет nested loop, нет индексов → важно, как устроены данные.

INNER JOIN LEFT JOIN RIGHT JOIN – работают также как в SQL

ANY INNER / ANY LEFT – взять любую подходящую строку, если их несколько. Это важно потому что ClickHouse не поддерживает full join

ASOF JOIN – Используется в временных данных, берёт ближайшее "раньше по времени" значение.

SEMI JOIN – это фильтрация по существованию

5. МУТАЦИИ, ВСТАВКА/ОБНОВЛЕНИЕ ДАННЫХ В CLICKHOUSE

Мутации пересборка частей данных в фоне.

Update – Через мутации. Построит новые части с измененными строками, а старые удалит.

GREENPLUM

1. ЧТО ТАКОЕ MPP

MPP (Massively Parallel Processing) это архитектура, где каждый узел обрабатывает часть данных параллельно и независимо.

Как устроено:

Есть один главный узел (мастер).

Есть много рабочих узлов (сегменты).

У каждого свои CPU, память, диск.

Все узлы соединены быстрой сетью.

2. ОПТИМИЗАЦИЯ ЗАПРОСОВ НА GREENPLUM

Правильный **DISTRIBUTED BY** – Распределяй таблицы по ключам, по которым часто происходит JOIN

Избегай **data motion** – Проверяй план через **EXPLAIN** ищи **Motion**

Проверяй план через **EXPLAIN** ищи **Motion**

Избегай **select**

Фильтры (WHERE) применяй до JOIN, чтобы не тянуть лишние строки

3. ПЕРЕКОСЫ В GREENPLUM

Data Motion в Greenplum (и других MPP-системах) – это процесс перемещения данных между сегментами (нодами) во время выполнения запроса.

Когда возникает:

Таблицы распределены по разным ключам, и нужно сделать **JOIN**

GROUP BY, ORDER BY, DISTINCT, которые требуют собрать или перестроить данные

Результат нужно вернуть на мастер (координатор)

Виды Data Motion:

1. Broadcast копирует строки со всех сегментов на каждый сегмент.

Возникает если 1 таблица маленькая

2. Redistribute (Hash Motion) пересылает строки по хешу нового ключа. Возникает при **JOIN** или **ORDER BY** по другому ключу.

3. **Gather** собирает данные с сегментов на мастер. Возникает при **ORDER BY**, **LIMIT** и **RETURN**

4. КАК РАСПЕДЕЛЯЮТСЯ ДАННЫЕ В GREENPLUM?

Когда ты создаёшь таблицу, ты указываешь **DISTRIBUTED BY ()**

Принцип работы

Greenplum считает хэш от значения

По хешу выбирает номер сегмента, куда записать строку

Каждая строка попадает на один конкретный сегмент

5. КАК ПРАВИЛЬНО ВЫБРАТЬ КЛЮЧ ДИСТРИБУЦИИ?

Правильный **DISTRIBUTED BY** - Распределяй таблицы по ключам, по которым часто происходит **JOIN**

SPARK

1. RDD VS DF VS DS

RDD (Разбитые Данные):

Как кирпичики - базовые блоки Spark.

Без схемы (не знаешь, что внутри объекта).

Требуют ручной оптимизации (медленнее).

Для сложных низкоуровневых задач.

DataFrame (Таблицы):

Как Excel-таблица со столбцами и названиями.

Со схемой (известны типы данных: число, строка).

Автоматическая оптимизация (быстрее, умный движок).

Для большинства задач (ETL, аналитика).

Dataset (Улучшенные Таблицы):

Как типизированные DataFrame.

Проверка типов на этапе компиляции (меньше ошибок).

Только для Scala/Java.

2. COALESCE VS REPARTITION

Repartition:

Увеличивает или уменьшает число партиций.

Всегда выполняет полный **shuffle** (дорогая операция).

Используй: Когда нужно увеличить параллелизм или равномерно распределить данные.

Coalesce:

Только уменьшает число партиций.

Избегает `shuffle`, объединяя партиции на тех же узлах.

Используй: Когда нужно уменьшить число партиций без `shuffle` (например, перед записью в `few files`).

3. SPILL, SHUFFLE

Shuffle дорогое перемешивание данных между узлами.

Когда: `groupBy`, `join`, `distinct`.

Как:

1. Write: Данные пишутся на диск.
2. Read: Читаются по сети для перегруппировки.

Проблемы:

Перекося (skew): Один ключ с большими данными.

Медленно и риск OOM.

Spill сброс данных из памяти на диск при нехватке RAM.

Когда:

Shuffle

Агрегации (`groupBy`, `join`)

Кэширование

Зачем: Чтобы избежать OOM (ошибка когда не хватает памяти) и обработать большие данные.

Минусы: Сильно медленнее (диск vs оперативная память).

Решение:

Увеличить число партиций

Настроить память (`spark.memory.fraction`)

Добавить RAM исполнителям

4. ФИЗИЧЕСКИЕ JOIN

1. Broadcast Hash Join (BHJ)

Когда: Одна из таблиц МАЛЕНЬКАЯ (меньше порога `spark.sql.autoBroadcastJoinThreshold`, по умолчанию 10 МБ).

Как: Малая таблица рассылается (broadcast) на все ноды. Быстро, т.к. избегает `shuffle`.

2. Sort Merge Join (SMJ)

Когда: Дефолтный `join` для больших таблиц. Обе стороны большие, и данные отсортированы по ключу `join`'а или требуется отсортированный вывод.

Как: Данные шарятся (shuffle), сортируются по ключу и мержаются. Эффективно для больших данных, но требует shuffle и сортировки.

3. Broadcast Nested Loop Join (BNLJ)

Когда: Если join condition НЕ равенство (например, `df1.value > df2.value`). Очень медленный, но иногда единственный вариант.

Как: Рассылает одну из таблиц и для каждой строки проверяет condition вложенным циклом. Избегайте, если возможно.

4. Cartesian Join (Cross Join)

Когда: Явно запрошен (через `crossJoin` или с условием `1=1`).

Декартово произведение очень дорого.

Как: Полный shuffle и объединение всех строк. Используйте только если осознанно.

5. ТРАНСФОРМАЦИИ, ДЕЙСТВИЯ, ЛЕНИВЫЕ ВЫЧИСЛЕНИЯ

Трансформации (например, `map`, `filter`):

Создают план вычислений (DAG)

Ленивые ничего не считают сразу

Действия (например, `count`, `save`, `show`):

Запускают выполнение всего плана

Возвращают результат или пишут данные

Ленивые вычисления:

Spark ждёт вызова действия, чтобы оптимизировать и выполнить весь план трансформаций разом

Экономит ресурсы и позволяет избежать лишних вычислений

КАК ЗАЛУТАТЬ ОФФЕРЫ

Я занимаюсь менторством. Средняя зп моих учеников 300к/месяц на руки



ПРЕДЛОЖЕНИЕ О РАБОТЕ

Уважаемый [REDACTED]
Приглашаем Вас присоединиться [REDACTED]

ОПИСАНИЕ РОЛИ
Должность: Старший эксперт
Подразделение: [REDACTED]
Руководитель: [REDACTED]
Место работы: [REDACTED]
Ожидаемая дата выхода на работу: [REDACTED]
Испытательный срок: 3 месяца
Ключевые задачи: [REDACTED]

СТРУКТУРА ВОЗНАГРАЖДЕНИЯ
Ваш ежемесячный фиксированный оклад составит 400 000 руб. (до уплаты НДФЛ).
Ежегодное увеличение оклада осуществляется в соответствии с политикой индексации заработной платы [REDACTED].
Премия:
Начиная с 2025 года, по итогам года Вы будете иметь право на получение премии в размере 4 ежемесячного(-ых) оклада(-ов) при условии 100%-го выполнения целевых показателей. Премия выплачивается в зависимости от достижения целевых показателей в первом квартале (март) года, следующего за отчетным, с учетом отработанного в течение года времени.

СОЦИАЛЬНЫЕ ПРОГРАММЫ
Вы будете иметь право на дополнительные льготы, предусмотренные [REDACTED] (включая, но не ограничиваясь):

- Оплата полиса медицинского страхования (ДМС).
- Оплата корпоративной мобильной связи.
- Предоставление служебного ноутбука.
- Компенсация расходов на поездки, связанные с исполнением служебных обязанностей.

Полная информация будет изложена в трудовом договоре и должностной инструкции.

**Райффайзен Банк**

Алексей,
будем очень рады, если вы примете наше предложение о работе и станете частью нашей команды.

Должность
разработчик
Developer

Подразделение
[REDACTED]

Формат работы
Дистанционный

Рабочий день
5 дней, 40 часов в неделю

Оклад до вычета налогов
400 000 руб.
Целевой размер годовой премии составляет 16% от годового дохода (пропорционально количеству отработанных месяцев, при условии работы в банке не менее 6 месяцев). Итоговая сумма премии зависит от результатов работы сотрудника, команды и банка

Трудовой договор
Постоянный

Испытательный срок
3 месяца

Место работы
[REDACTED]

Лист согласования (job offer)

ФИО [REDACTED]
Подразделение [REDACTED]
Должность **Технический руководитель по направлению data-инженерии**

Срок действия трудового договора: на неопределённый срок
выберите нужное значение

Дата выхода на работу 2025 г.
выберите нужное значение

Продолжительность испытательного срока 3 месяца
выберите нужное значение

Дата окончания испытательного срока 2025 г.

Стартовая зарплата на испытательный срок 402 298 руб. gross

Повышение зарплаты предусмотрено через месяцев

Зарплата после повышения

Зарплата после окончания исп. срока 402 298 руб. gross + 15% квартальная премия

Рост з/п в дальнейшем:

– запланировано повышение з/п через мес. до руб. gross, руб. net., при условии

– в обычном порядке;

– другое

ХОЧЕШЬ ТАКЖЕ - ЗАХОДИ НА ОБУЧЕНИЕ

Если ты сейчас готовишься к собеседам на дата инженера или хочешь стать им, то приходи ко мне на обучение.

За символическую предоплату я доведу тебя до трудоустройства за ближайшие 3-4 месяца.

Или если ты уже ходишь по собеседам и не можешь найти работу – тоже пиши. Я помогу с легендой и резюме.



**ЗАПИСАТЬСЯ НА
ОБУЧЕНИЕ**